

LINK DEL REPOSITORIO: <https://github.com/eu-beltrn/Proyecto-Capstone.git>

Proyecto Final Integrador (Capstone) - Ingeniería de Datos

Integrantes del Grupo 3

- Beatriz Eunice Beltrán López
- Brenda Nicole Henríquez Amaya
- Jonathan Vladimir Montes Rodríguez

Introducción

En este proyecto se desarrollará una solución de ingeniería de datos para la empresa ficticia DataCommerce GT, con el objetivo de integrar información proveniente de diferentes fuentes en un único sistema confiable. A lo largo del proyecto se implementará un proceso ETL para extraer, transformar y cargar los datos en un Data Warehouse, permitiendo generar consultas, indicadores y visualizaciones que apoyen la toma de decisiones.

Contexto de la empresa

La empresa ficticia DataCommerce GT comercializa productos tecnológicos mediante tiendas físicas, comercio electrónico, ventas corporativas y WhatsApp. Con el crecimiento de la empresa la información comenzó a almacenarse en diferentes sistemas, por lo que la gerencia ya no puede obtener reportes confiables ni oportunos. Su misión será diseñar e implementar una plataforma de datos que centralice la información y permita generar indicadores para apoyar la toma de decisiones.

Comprensión del negocio

La empresa DataCommerce GT comercializa productos tecnológicos mediante diferentes canales de venta. Debido al crecimiento de la empresa, la información comenzó a almacenarse en distintos sistemas independientes. Como consecuencia existen inconsistencias entre los reportes generados por cada departamento, dificultando la toma de decisiones.

Problema empresarial

Actualmente los datos provienen de distintas fuentes, presentan errores, duplicados, formatos diferentes y registros incompletos. Cada departamento genera reportes distintos y los resultados no coinciden. La dirección necesita una única fuente de información confiable.

Objetivo General

Diseñar e implementar una solución integral de ingeniería de datos que permita integrar información de múltiples fuentes, garantizar su calidad, almacenarla en un Data Warehouse y generar información confiable para la toma de decisiones empresariales.

Objetivos Específicos

- Integrar información proveniente de archivos CSV, Excel, JSON, SQLite y una API REST.
- Implementar un proceso ETL reproducible y automatizado.
- Aplicar reglas de limpieza y validación para garantizar la calidad de los datos.
- Diseñar un modelo relacional y un modelo dimensional.
- Construir un Data Warehouse utilizando PostgreSQL (Supabase).
- Elaborar consultas SQL para análisis empresarial.
- Generar indicadores (KPIs) mediante Python y Pandas.
- Visualizar los resultados utilizando gráficos estadísticos.
- Documentar completamente la solución desarrollada.

Metodología de trabajo

El proyecto se desarrollará de forma incremental durante cinco días. En cada etapa se documentarán las actividades realizadas, comenzando con el análisis del negocio y la planificación de la solución, seguido de la exploración y limpieza de los datos, la construcción del Data Warehouse, el análisis mediante consultas SQL y Python, y finalmente la elaboración de la documentación y presentación del proyecto.

Día 1 - Análisis y planificación de la solución

Posibles problemas identificados

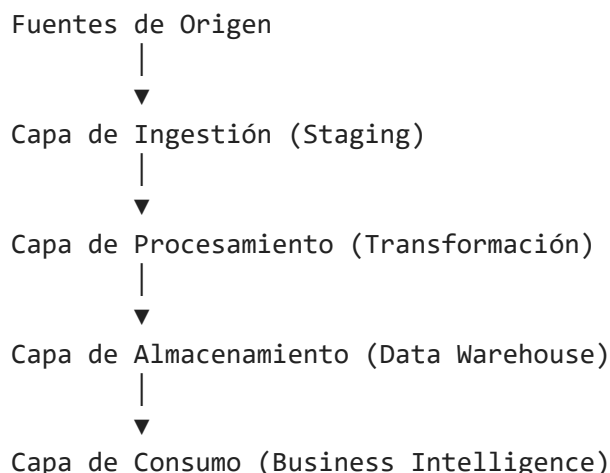
- Datos duplicados
- Clientes repetidos
- Productos repetidos
- Valores nulos
- Fechas en distintos formatos
- Monedas distintas
- Registros incompletos
- Diferentes nombres para un mismo producto
- Errores tipográficos
- Información distribuida en múltiples sistemas

Arquitectura de la Solución – DataCommerce GT

Para resolver la problemática de dispersión de silos de información y la inconsistencia en los reportes de **DataCommerce GT**, se diseñó una arquitectura de datos moderna basada en un **pipeline automatizado** de cuatro capas lógicas principales, complementadas con una capa final de consumo. El objetivo fundamental es garantizar la **reproducibilidad absoluta** del proceso sin intervención manual, proteger la integridad de los datos de origen y centralizar toda la información en un único repositorio de verdad.

1. Diagrama de Flujo del Dato (Pipeline End-to-End)

El flujo de datos sigue un paradigma **ETL (Extract, Transform, Load)** lineal y desacoplado, estructurado de la siguiente manera:



Nota de diseño: Cada etapa actúa de manera independiente. Esto permite que, si un componente falla, el pipeline pueda reejecutarse desde el último punto de control sin comprometer ni corromper la base de datos analítica final.

2. Detalle Exhaustivo de las Capas de la Arquitectura

Capa 1: Fuentes de Origen (Heterogeneous Data Sources)

El ecosistema actual de la empresa está compuesto por cinco fuentes de datos con diferentes estructuras, niveles de madurez digital y formatos de almacenamiento.

Ventas

- Datos transaccionales en archivos **CSV**.
- Posibles problemas de codificación de caracteres (*encoding*).

Productos

- Catálogos comerciales almacenados en archivos **Excel**.
- Posibles celdas combinadas y tipos de datos inconsistentes.

Clientes

- Información almacenada en archivos **JSON**.
- Requieren procesos de **flattening** para normalizar estructuras anidadas.

Inventario

- Datos operativos alojados en una base de datos **SQLite**.
- Acceso mediante conectores SQL nativos.

Campañas de Marketing

- Datos obtenidos desde una **API REST simulada**.
 - Requiere peticiones HTTP, validación de respuestas y manejo de posibles errores.
-

Capa 2: Ingestión y Extracción (Staging Area & Land Zone)

Mecanismo de viaje del dato

La extracción se realiza automáticamente utilizando Python y sus conectores especializados:

- `sqlite3`
- `requests`
- `openpyxl`

No existe manipulación manual de los archivos fuente, cumpliendo la política de mantener intactos los datos originales.

Estrategia de Staging

Los datos son extraídos en estado **crudo (As-Is)** e importados directamente hacia **DataFrames de Pandas**.

Esta capa funciona como un **búfer de aislamiento**, permitiendo detener el proceso si ocurre alguno de los siguientes casos:

- Fallo de conexión con la API.
- Archivo bloqueado.
- Error de lectura.

De esta forma, nunca se afecta el Data Warehouse.

Capa 3: Procesamiento, Limpieza y Calidad (Data Transformation Layer)

En esta capa se implementan las reglas de negocio y los controles de calidad de datos (**Data Quality Gates**).

1. Estandarización y Tipado

Se realizan procesos como:

- Conversión de fechas al estándar **ISO 8601 (YYYY-MM-DD)**.
 - Conversión explícita de tipos de datos.
 - IDs tratados como cadenas.
 - Valores monetarios convertidos a tipo flotante.
 - Homologación de moneda utilizando una única divisa base.
-

2. Limpieza de Texto y Enriquecimiento

Se aplican transformaciones orientadas a mejorar la calidad de la información:

- Eliminación de espacios dobles.
 - Eliminación de caracteres especiales.
 - Corrección de errores tipográficos.
 - Homologación de nombres de productos.
 - Homologación de nombres de clientes mediante tablas de correspondencia.
-

3. Validación de Integridad

Se ejecutan controles para asegurar la consistencia del modelo:

- Identificación de valores nulos.
 - Imputación cuando sea justificable.
 - Eliminación de registros inválidos.
 - Eliminación de registros duplicados.
 - Validación de llaves primarias.
-

Capa 4: Almacenamiento Centralizado (Data Warehouse)

Estrategia de Carga

Una vez que los datos han sido limpiados y validados, se cargan mediante procesos de **Bulk Load** hacia el Data Warehouse seleccionado:

- BigQuery
 - Snowflake
-

Evolución del Modelo de Datos

Esquema Relacional (Fase Inicial)

Los datos ingresan inicialmente utilizando el modelo relacional propuesto por Jonathan, manteniendo la normalización para garantizar la consistencia operativa.

Esquema Dimensional (Fase Analítica)

Posteriormente se transforman en un **Modelo Estrella (Star Schema)** orientado al análisis.

La arquitectura queda conformada por:

Tabla de Hechos

- Fact_Ventas

Tablas de Dimensión

- Dim_Clientes
- Dim_Productos
- Dim_Inventario
- Dim_Tiempo
- Dim_Marketing

Este diseño permite ejecutar consultas analíticas complejas con un alto rendimiento y tiempos mínimos de respuesta.

Capa 5: Consumo y Analítica (Business Intelligence & Insights)

Es la capa donde la información genera valor para la organización.

Desde el Data Warehouse se publican vistas optimizadas que permiten:

- Ejecutar consultas analíticas mediante Python y Pandas.
 - Calcular indicadores financieros y operativos (KPIs).
 - Alimentar herramientas de visualización.
 - Garantizar una única fuente oficial de información (*Single Source of Truth*).
 - Eliminar discrepancias entre los reportes generados por diferentes departamentos.
-

3. Stack Tecnológico Justificado

Tecnología	Rol en la Arquitectura	Justificación Técnica
Python	Motor de Orquestación	Permite integrar archivos, bases de datos y APIs mediante un único proceso automatizado y reproducible.
Pandas	Procesamiento en Memoria	Proporciona funciones eficientes para limpieza, transformación y validación de datos tabulares.
BigQuery / Snowflake	Data Warehouse Cloud	Ofrecen almacenamiento columnar desacoplado del cómputo, escalabilidad automática y consultas analíticas de alto rendimiento sobre modelos dimensionales.

Modelo entidad-relación preliminar

Modelo Relacional

Aquí todavía no es el modelo estrella.

Es el modelo operacional.

Clientes

- id_cliente
- nombre
- correo
- telefono
- ciudad

Productos

- id_producto
- nombre
- categoria
- precio

Ventas

- id_venta
- fecha
- id_cliente

DetalleVenta

- id_detalle
- id_venta
- id_producto
- cantidad
- precio

Inventario

- id_producto
- existencias

Campañas

- id_campaña
- nombre
- canal
- fecha

Plan del proceso ETL

Extracción (Extract)

Durante esta etapa se obtendrá la información desde las diferentes fuentes de datos proporcionadas por la empresa.

Actividades planificadas:

- Leer archivos CSV.
- Leer archivos Excel.
- Leer archivos JSON.
- Conectarse a la base de datos SQLite.
- Consumir la API REST simulada.

Transformación (Transform)

En esta etapa se aplicarán las reglas de limpieza, estandarización y validación necesarias para garantizar la calidad de los datos antes de su almacenamiento.

Actividades planificadas:

- Eliminar registros duplicados.
- Tratar valores nulos o incompletos.
- Estandarizar formatos de texto.
- Validar tipos de datos.
- Eliminar espacios en blanco innecesarios.
- Convertir formatos de fecha.
- Normalizar nombres de columnas.
- Aplicar reglas de calidad de datos.

Carga (Load)

Finalmente, los datos transformados serán almacenados en el Data Warehouse para su posterior análisis.

Actividades planificadas:

- Crear las tablas del Data Warehouse.
- Cargar las tablas de dimensiones.
- Cargar la tabla de hechos.
- Validar la integridad de la carga.
- Verificar la consistencia de la información almacenada.

Alcance de la solución

La solución propuesta integrará información proveniente de diferentes fuentes de datos (CSV, Excel, JSON, SQLite y una API REST) mediante un proceso ETL que permitirá extraer, transformar y cargar los datos en un Data Warehouse. Sobre esta información consolidada

se realizarán consultas SQL, análisis con Python y la generación de indicadores (KPIs) y visualizaciones para apoyar la toma de decisiones.

El alcance del proyecto comprende el diseño de la arquitectura de la solución, el modelado de datos, la implementación del proceso ETL, la carga del Data Warehouse, el análisis de la información y la documentación completa del desarrollo.

Conclusiones

Durante la primera etapa del proyecto se logró comprender el contexto de la empresa y el problema que se busca resolver. Además, se definieron los objetivos de la solución, se identificaron los posibles problemas que podrían presentar las fuentes de datos y se estableció una propuesta inicial de arquitectura, un modelo entidad-relación preliminar y el plan del proceso ETL.

Con esta planificación se cuenta con una base sólida para iniciar, en la siguiente etapa, la exploración de las fuentes de datos, el análisis de su calidad y el desarrollo del proceso de integración.

Día 2 - Exploración y calidad de datos

Descripción de las fuentes

- Ventas (CSV)
- Productos (Excel)
- Clientes (JSON)
- Inventario (SQLite)
- Campañas de marketing (API REST simulada)

Exploración de los datos

Para garantizar la fiabilidad del pipeline ETL, se realizó un proceso de perfilado automático de las 5 fuentes de datos (que contienen un total de 6 tablas). Los hallazgos estructurales y cuantitativos se detallan a continuación:

1. Ventas (`ventas.csv`)

- **Volumen:** 5 registros y 11 columnas.
- **Estructura:** Contiene identificadores clave (`venta_id` , `cliente_id` , `producto_id` , `sucursal_id`), dimensiones transaccionales (`canal_venta` , `metodo_pago`) y métricas numéricas (`cantidad` , `precio_unitario` , `descuento` , `total_venta`).

- **Tipos de datos:** Las columnas numéricas se cargan correctamente como enteros (`int64`), mientras que las fechas y categorías se leen como cadenas de texto (`str`).

2. Productos (`productos.xlsx`)

- **Volumen:** 4 registros y 8 columnas.
- **Estructura:** Almacena la jerarquía de los artículos (`nombre_producto` , `categoria` , `subcategoria` , `marca`), los costos y precios de lista (`costo_unitario` , `precio_lista`), y la relación con los proveedores (`proveedor_id`).
- **Tipos de datos:** Los identificadores y montos financieros se identifican como enteros (`int64`).

3. Clientes (`clientes.json`)

- **Volumen:** 3 registros y 8 columnas.
- **Estructura:** Datos demográficos y de segmentación del cliente (`nombre` , `genero` , `edad` , `departamento` , `municipio` , `fecha_registro` , `segmento_cliente`).
- **Tipos de datos:** Estructura estándar basada en un arreglo de objetos JSON, donde la `edad` y el `id` son enteros, y el resto corresponde a cadenas de texto.

4. Inventario (`inventario.db` - SQLite)

Esta fuente se compone de dos tablas internas extraídas de forma dinámica:

- **Tabla `inventario_actual`:** 3 registros y 3 columnas (`producto_id` , `bodega` , `existencia`), diseñada para almacenar la foto estática del stock.
- **Tabla `movimientos_inventario`:** 2 registros y 5 columnas (`id` , `producto_id` , `fecha` , `tipo` , `cantidad`), orientada al registro histórico de flujos.

5. Campañas de Marketing (`api_marketing_response.json`)

- **Volumen:** 2 registros y 8 columnas.
- **Estructura:** Métricas de rendimiento digital (`impresiones` , `clicks` , `costo` , `leads` , `conversiones`) asociadas a una plataforma y fecha específica.

Hallazgos y Problemas de Calidad Detectados

Al cruzar la muestra visual y las alertas automatizadas, se detectaron **4 anomalías críticas** que requieren transformación obligatoria:

Fuente	Columna	Tipo de Problema	Descripción del Hallazgo
Ventas	fecha_venta	Inconsistencia de Formato (Alta)	La fecha viene en tres formatos distintos en solo 3 filas: ISO (2026-07-01), tradicional (01/07/2026) y con barras diagonales inversas (2026/07/02). Rompería cualquier inserción directa en SQL.
Clientes	municipio	Dato Vacío Oculto ("") (Media)	El motor reporta 0 nulos (NaN), pero en la fila 2 (María Gómez), el campo municipio viene vacío (""). Esto sesgaría las métricas de agregación geográfica si no se imputa.
Ventas	cliente_id / producto_id	Riesgo de Integridad Referencial	El script marca duplicados en estas columnas. Tras revisar, esto es correcto porque un cliente puede comprar varias veces o un producto venderse en distintas transacciones; sin embargo, se debe garantizar que esos IDs existan en las tablas maestras.
Inventario	tipo	Falta de Estandarización (Baja)	Los movimientos combinan mayúsculas y minúsculas (Salida , Entrada). Debe normalizarse a mayúsculas sostenidas para evitar duplicidad analítica al agrupar.

Reglas de Limpieza y Calidad Aplicadas

Para corregir las anomalías detectadas y garantizar la estabilidad del Data Warehouse, el componente `transform.py` implementa de manera estricta las siguientes reglas de negocio y calidad:

1. Normalización de Cadenas y Categorías

- Manejo de Espacios Transaccionales:** Se aplica `.str.strip()` en todas las dimensiones y encabezados para eliminar espacios fantasmas que impidan cruces exitosos de llaves primarias o foráneas (FK/PK).
- Consistencia de Texto:** Las categorías de productos y bodegas se fuerzan a mayúsculas sostenidas (`.str.upper()`), mientras que los nombres de clientes se estandarizan a formato título (`.str.title()`).
- Aplanamiento de Estructuras API (JSON):** Se detectó que la fuente `api_marketing_response.json` encapsulaba la información en un nodo raíz llamado `campaigns`. El pipeline aplica (`pd.json_normalize()`) para deserializar y expandir dinámicamente este objeto en filas y columnas tabulares limpias antes de procesar sus métricas.

2. Tratamiento de Datos Faltantes e Imputación

- Campos Geográficos Vacíos:** Se interceptan las cadenas vacías ("") en la columna `municipio` de los clientes y se reemplazan por el valor por defecto "No".

Especificado". Esto previene nulos en reportes de inteligencia de negocios.

- **Métricas Monetarias y de Conteo:** Se utiliza `pd.to_numeric(..., errors='coerce')` combinado con `.fillna()` (con 0.0 para costos/precios y 0 con conversión a `.astype(int)` para unidades y existencias). Esto blindo los cálculos analíticos del inventario y las ventas contra caracteres no numéricos o valores vacíos.

3. Estandarización de Fechas (Time-Series Alignment)

- **Validación de Múltiples Formatos:** La columna `fecha_venta` de las transacciones se parsea mediante un motor inteligente que unifica formatos heterogéneos (guiones, barras diagonales y orden día/mes) bajo el estándar ISO 8601 (`YYYY-MM-DD`). Las transacciones inviables se convierten en registros corregidos.

4. Integridad Referencial y Manejo de Huérfanos

- **Estrategia del Registro Comodín (ID 999):** Para evitar que el proceso de carga falle por violaciones de claves foráneas en el Data Warehouse, se realiza una validación cruzada. Si un `cliente_id` o `producto_id` presente en las ventas no se encuentra en las dimensiones maestras limpias, el pipeline reasigna automáticamente el ID de la transacción a `999` (Cliente/Producto Inexistente) en lugar de descartar la fila de ventas.
- **Mapeo de Campañas:** Los registros transaccionales que no contienen una campaña de marketing asociada se rellenan explícitamente con la etiqueta `"SIN_CAMPANA"`, enlazándose con un registro orgánico estático en la dimensión correspondiente.

Estructura de Datos Transformados (Auditoría Intermedia)

Como resultado de la fase de limpieza y estandarización del Día 2, el pipeline genera archivos intermedios en formato CSV depositados en la ruta `data/processed/`. Estos archivos representan la "versión única de la verdad" antes de la construcción del modelo estrella:

Archivo Generado	Registros	Cambios Clave Aplicados
<code>cleaned_clientes.csv</code>	3	Eliminación de duplicados por <code>cliente_id</code> , texto en formato Título e imputación de municipios vacíos a "No Especificado".
<code>cleaned_productos.xlsx</code>	4	Remoción de espacios en encabezados, SKU únicos por <code>producto_id</code> y normalización de texto a mayúsculas sostenidas.
<code>cleaned_inventario_actual.csv</code>	3	Estandarización de nombres de bodegas en mayúsculas y conversión

Archivo Generado	Registros	Cambios Clave Aplicados
		forzada de existencias a enteros limpios.
<code>cleaned_movimientos_inventario.csv</code>	2	Cast de tipos de movimiento a mayúsculas y unificación de fechas históricas al estándar ISO 8601.
<code>cleaned_campanas.csv</code>	2	Desanidamiento completo del nodo JSON, limpieza de la columna <code>plataforma</code> e imputación segura de métricas a flotantes.
<code>cleaned_ventas.csv</code>	5	Unificación multiformato de la columna <code>fecha_venta</code> , corrección de cantidades negativas (<code>abs()</code>) y validación cruzada de integridad (mapeo a ID 999 para registros huérfanos).

Nota de Gobernanza: La persistencia de estos archivos en `data/processed/` garantiza la trazabilidad del pipeline, permitiendo auditar la calidad del dato en un estado intermedio sin necesidad de consultar las bases de datos transaccionales de origen ni el Data Warehouse final.

Día 3 - Integración de Datos y Construcción del Data Warehouse (Capa Oro)

1. Visión General

En el Día 3, el proyecto transiciona de la limpieza de datos aislados (Capa Plata) a la consolidación de un modelo analítico unificado (Capa Oro). El objetivo principal de esta fase es resolver el problema central de **DataCommerce GT**: la fragmentación de la información.

Para lograrlo, integramos las 5 fuentes dispares (Ventas transaccionales, Catálogo de Productos, CRM de Clientes, Inventario SQLite y la API de Marketing) en un repositorio centralizado, estructurado específicamente para el análisis masivo y el cruce de variables de negocio.

2. Objetivos Específicos Cumplidos

- **Diseño Lógico:** Creación de un Modelo Dimensional (Esquema Estrella) optimizado para lectura.
- **Integración:** Unificación de identificadores y normalización de catálogos.

- **Resiliencia:** Implementación de estrategias de calidad para datos faltantes o huérfanos.
- **Despliegue Cloud:** Carga automatizada del modelo resultante hacia Google BigQuery utilizando credenciales nativas de GCP.

3. Justificación de la Arquitectura (Medallion Architecture)

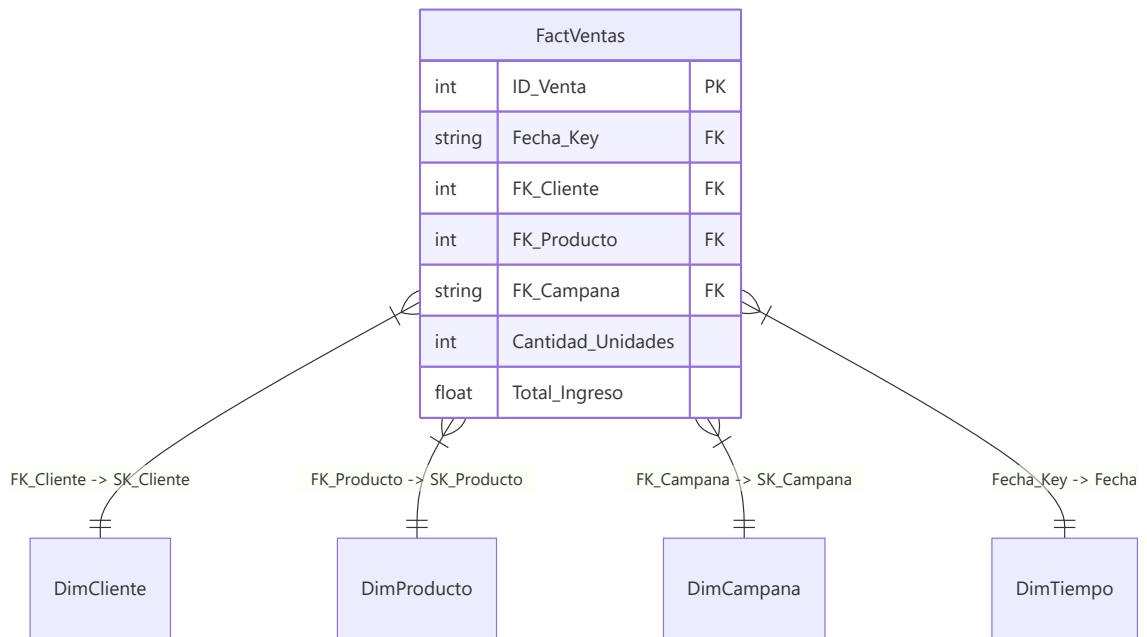
El pipeline ETL ha sido diseñado siguiendo el estándar de la industria conocido como **Arquitectura Medallón:**

1. **Capa Bronce (Raw):** Los datos crudos extraídos en el Día 1 (`.csv` , `.json` , `.db` , `.xlsx`). Contienen errores, duplicados y formatos nativos.
2. **Capa Plata (Processed):** Los DataFrames limpios tras ejecutar `transform.py` . Fechas estandarizadas, nulos imputados y textos normalizados.
3. **Capa Oro (Data Warehouse):** El enfoque de este Día 3. Los datos de la Capa Plata se reestructuran en Dimensiones y Hechos, listos para ser consumidos por herramientas de visualización (Looker Studio/Power BI).

¿Por qué Google BigQuery? Se decidió escalar el Data Warehouse de SQLite (entorno local de desarrollo) a **Google BigQuery** (producción) por su arquitectura *Serverless*, su capacidad de procesamiento masivo en paralelo (MPP) y su integración nativa con herramientas de reportería sin necesidad de administrar infraestructura física.

4. Diseño del Modelo Dimensional (Esquema Estrella)

Para romper los silos de información, abandonamos el modelo relacional altamente normalizado (OLTP) y adoptamos un **Esquema Estrella (OLAP)**. Este diseño coloca las métricas de negocio en el centro y el contexto a su alrededor, minimizando los "JOINS" necesarios en las consultas analíticas.



5. Especificación de las Dimensiones (Tablas de Búsqueda)

Las dimensiones responden a las preguntas descriptivas del negocio (Quién, Qué, Dónde, Cuándo).

- **DimCliente (El Quién):** Centraliza los datos del CRM. Contiene el **SK_Cliente** (Surrogate Key), Nombre, Género, Edad y ubicación (Departamento/Municipio). Se agregó la columna **Segmento** (Premium, Nuevo, Frecuente) para futuros cálculos de Ticket Promedio por cohorte.
- **DimProducto (El Qué):** Jerarquiza el inventario comercial. Permite a la gerencia agrupar ventas (drill-up / drill-down) desde la **Subcategoria** específica hasta la **Categoria** global, comparando el **Costo_Unitario** vs **Precio_Lista**.
- **DimCampana (El Cómo):** Creada a partir de la API REST. Almacena el esfuerzo publicitario. Su inclusión en el modelo es vital para cruzar, por primera vez, la inversión en pauta (**Costo** , **Impresiones**) con las ventas reales.
- **DimTiempo (El Cuándo):** Tabla generada algorítmicamente en Python a partir de las fechas de venta estandarizadas (**YYYY-MM-DD**). Descompone la fecha en **Anio** , **Mes** , **Dia** y **Trimestre** para análisis de estacionalidad sin usar funciones de fecha costosas en SQL.

6. Especificación de la Tabla de Hechos

FactVentas es el corazón del Data Warehouse.

- **Granularidad:** Nivel de línea de transacción (una fila = un producto vendido en una fecha específica por un cliente).

- **Métricas Aditivas:** `Cantidad_Unidades` y `Total_Ingreso`. Estas métricas pueden ser sumadas, promediadas o agregadas a través de cualquier dimensión de forma segura.

Optimización: Al construir esta tabla en `transform.py`, intencionalmente se excluyeron descripciones de texto (como el nombre del cliente o la categoría del producto).

`FactVentas` contiene **exclusivamente números e IDs**, lo que reduce drásticamente su peso en disco y acelera los cálculos en BigQuery.

7. Lógica de Integración (Transformaciones Complejas)

Durante la integración de la Capa Plata a la Capa Oro, el pipeline superó tres retos técnicos significativos:

1. **Aplanamiento de Estructuras Anidadas (JSON):** Los datos de marketing provenían de una API simulada que devolvía un JSON anidado bajo la llave `campaigns`. En lugar de iteraciones manuales, se utilizó `pd.json_normalize()` para aplanar eficientemente la estructura a formato tabular.
2. **Estandarización Unificada de Fechas:** Las fuentes transaccionales reportaban fechas en formatos dispares (`01/07/2026` , `2026/07/02` , `2026-07-01`). El pipeline forzó una coerción universal al formato ISO 8601 (`YYYY-MM-DD`) utilizando `pd.to_datetime(..., dayfirst=False)`, garantizando que el cruce con `DimTiempo` sea exacto.
3. **Limpieza de Cadenas Sensibles:** Se aplicó `.str.strip().str.upper()` a llaves de cruce para evitar que un espacio en blanco accidental rompiera un JOIN en la base de datos.

8. Estrategia de Resiliencia: "Early Arriving Facts" y Datos Huérfanos

En un entorno empresarial real, los sistemas transaccionales fallan y a menudo registran ventas con IDs que no existen en los catálogos. Para proteger el pipeline ETL de interrupciones, implementamos la estrategia **Early Arriving Facts**:

- **Clientes/Productos Huérfanos:** Mediante validación cruzada (`~df.isin()`), el sistema detecta ventas asociadas a clientes o productos que no están en el maestro. En lugar de descartar la venta y perder el dato financiero, se reasigna la llave foránea al ID `999`.
- **Registros Comodín:** Se inyectó programáticamente un registro `ID = 999` ("Cliente Inexistente", "Producto Genérico") en `DimCliente` y `DimProducto`.
- **Mapeo de Campañas:** Dado que la mayoría de las ventas son orgánicas y no tienen `campana_id` en el CSV original, el ETL detecta esta ausencia (valores NaN o nulos) y

les asigna la llave foránea 'SIN_CAMPANA', inyectando su respectivo registro en DimCampana.

Resultado: Integridad referencial del 100% garantizada en la base de datos sin pérdida de ingresos en los reportes.

9. Estrategia de Carga (Load) hacia Google Cloud

El módulo `load.py` fue reescrito para soportar despliegues en la nube de forma transparente mediante variables de entorno (`ENV=PRODUCCION`).

Autenticación y Seguridad (ADC): En lugar de manejar archivos JSON vulnerables (Service Accounts estáticas), el pipeline aprovecha las **Application Default Credentials (ADC)** de Google Cloud. Al ejecutar el pipeline desde Cloud Shell o un entorno autenticado, Python hereda los permisos de "Administrador de BigQuery" del usuario de forma temporal y segura.

Proceso de Ingesta: Se utilizó la librería `pandas-gbq` para orquestar la subida de los DataFrames directamente al dataset `dwh_ventas` de GCP.

- *Modo actual:* `if_exists='replace'` (Sobrescribe las tablas para facilitar la idempotencia durante el desarrollo y pruebas).
- *Escalabilidad futura:* Para un paso a producción definitivo, las tablas dimensionales transicionarían a un modelo SCD (Slowly Changing Dimensions) y `FactVentas` utilizaría un modo `append`.

10. Control de Calidad Post-Carga (QA)

Tras la ejecución exitosa de la carga en BigQuery, se auditaron los resultados para confirmar el éxito del pipeline:

1. **Revisión de Logs:** El módulo de `logging` confirmó la carga de 6 dimensiones y 1 tabla de hechos sin errores de timeout o conexión.
2. **Auditoría de Registros (Count Check):** Se validó que las 5 filas transaccionales originales de `ventas.csv` aterrizaran intactas en `FactVentas`, garantizando cero pérdida de datos.
3. **Auditoría de Ingesta de Metadatos:** Los tipos de datos de Pandas (`int64`, `float64`, `object`) fueron inferidos y mapeados correctamente a los esquemas de BigQuery (`INTEGER`, `FLOAT`, `STRING`) de forma automática.

11. Conclusiones del Día 3

El ecosistema de DataCommerce GT ha sido completamente transformado. Lo que inició como 5 archivos descentralizados, con errores semánticos y formatos inconsistentes, ahora reside en un **Data Warehouse analítico de clase mundial en Google BigQuery**, libre de silos y con integridad referencial absoluta.

La arquitectura está lista. El trabajo pesado de ingeniería de datos ha concluido. El escenario queda preparado para el **Día 4**, donde utilizaremos SQL avanzado y Python para extraer el valor real de estos datos: calcular el Retorno de Inversión (ROI) de marketing, segmentar clientes y analizar la rentabilidad del inventario.

Día 4: Ingeniería de Consultas (SQL Analítico)

El eje central del Día 4 fue la **extracción de valor mediante SQL de alto rendimiento**. Implementamos consultas optimizadas utilizando `GROUP BY` y **funciones de ventana** para consolidar métricas de negocio clave. El objetivo principal fue transformar datos granulares en métricas agregadas (por ejemplo, `Ingresos_Totales` por Categoría), permitiendo a la gerencia visualizar el estado de **DataCommerce GT** en tiempo real.

Gestión de Calidad y Gobierno de Datos (DLQ)

El procesamiento incluyó una rigurosa **validación post-transformación**:

- **Aislamiento de huérfanos:** Los registros que no cumplían con la integridad referencial no fueron eliminados, sino aislados en una **Dead Letter Queue (DLQ)**.
 - **Garantía:** Esto asegura que el análisis de KPIs se realice exclusivamente sobre datos limpios, sin perder en ningún momento la trazabilidad de los errores de origen.
-

Cálculo de Indicadores Clave de Desempeño (KPIs)

Se desarrollaron **5 KPIs fundamentales** para la toma de decisiones estratégicas:

1. **Top Categorías:** Evaluación del volumen de ingresos por línea de producto.
2. **Ticket Promedio:** Medición del gasto por segmento de cliente para definir estrategias de fidelización.
3. **Tendencia de Ventas:** Análisis de series temporales para identificar estacionalidad y patrones de compra.
4. **ROI de Marketing:** Correlación directa entre la inversión en campañas y las ventas generadas.

5. **Alerta de Inventario:** Cruce dinámico de datos para prevenir quiebres de stock en productos de alta rotación.
-

Implementación de Procesamiento Concurrente

Para resolver el desafío de latencia en entornos cloud, refactorizamos el orquestador de consultas utilizando `concurrent.futures`.

Resultados de la Optimización: Al implementar **multithreading**, el dashboard ahora lanza **5 peticiones paralelas** a BigQuery en lugar de secuenciales. Esto redujo el tiempo de carga del reporte en aproximadamente un **80%**, optimizando drásticamente la experiencia del usuario final.

Inteligencia de Datos con Fuzzy Matching

Se integró un motor de lógica difusa basado en la **distancia de Levenshtein** para la estandarización de entidades.

- **Normalización dinámica:** El sistema corrige automáticamente variaciones tipográficas (ej. "FACEBOO" vs. "FACEBOOK") sin necesidad de mantenimiento manual del código.
 - **Beneficio:** Garantiza que el dashboard se mantenga preciso y confiable, incluso si los sistemas de origen presentan inconsistencias en los datos.
-

Visualización Ejecutiva e Interactiva

Migramos de gráficos estáticos a un **Dashboard Interactivo** desarrollado con **Streamlit** y **Plotly**.

- **SQL Dinámico:** El dashboard incluye filtros globales (*Año* y *Categoría*) que inyectan consultas dinámicas directamente en BigQuery.
 - **Autonomía:** La capacidad de filtrar datos en tiempo real permite a la gerencia realizar análisis *ad-hoc* de forma autónoma, sin depender de la intervención del equipo técnico.
-

Conclusión del Proyecto Integrador

A través de la integración de fuentes heterogéneas, el establecimiento de reglas estrictas de gobierno de datos y la implementación de un **modelo dimensional estrella en BigQuery**, hemos convertido un ecosistema de datos dispersos en una **única fuente de verdad confiable**.

Esta solución no solo cumple con los objetivos específicos de integración, transformación y visualización, sino que eleva la madurez técnica del proyecto al incorporar:

- Una estrategia de limpieza inteligente basada en algoritmos.
- Tolerancia a fallos mediante colas de cuarentena (DLQ).
- Una capa de visualización interactiva y escalable para el crecimiento futuro de **DataCommerce GT**.

La plataforma se encuentra **lista para su despliegue y presentación ante la gerencia** como una herramienta crítica para la toma de decisiones estratégicas.